Title:	**Group Project**
Course Code:	**COMP3901**
Credits:	**3**
Level:	**3**
Prerequisite:	**COMP2140 – Introduction to Software Engineering** **COMP2211 – Analysis of Algorithms** **Any 6 credits of Level 2 or 3 Computing Courses**
Semester:	**2 or 3**
Requirement:	**B. Sc. Computer Science and Information Technology**

Rationale:

COMP3901 is the group project course in the Computer Science curriculum. It is a required course for all students majoring in Computer Science and Information Technology. It is intended to be a capstone course that will bring together many of the topics that were covered in the rest of the curriculum. For this reason, we expect that students take this course in their final year, or at least after they have completed all of the core courses of the curriculum.

Course Description:

1 Introduction

In this course, students will identify a project they would like to work on, and be assigned to a supervisor who is able to provide guidance on the specific topic they have chosen. The scope of acceptable projects is wide open. Students are encouraged to approach this course with an open mind, and to be creative in coming up with their projects. In coming up with a project idea, you are encouraged to think about problems that need solving, rather than technologies that you know how to apply. Your supervisor will help you to scope your project appropriately, and to provide technical assistance with some aspects of your design and implementation. However, do not expect that your supervisor will know all of the answers to all of your questions. Learning specific details that are rarely known to other persons in the computing field is something you should learn to expect when you become fully immersed in a project; and you will find that it can be a tremendously satisfying experience. One way to view COMP3901 is as an opportunity to learn about a computing topic that was not covered to your satisfaction within the rest of the curriculum; you are encouraged to make the most of that opportunity.

1.1 How It Works

There are no formal lectures for COMP3901, and it is not assessed by the traditional final examination by which most of the other courses offered by the department are assessed. The department announces a date within the first week of registration for a new semester when the course will officially begin with its first meeting. The prescribed schedule of events is set out in Table 1.

The schedule is modified a little during the Summer because there are only nine (9) teaching weeks during the Summer. In that case, the first three items are completed within two weeks (the first two are done on the first meeting, and students use the first week to come up with their proposals). The midterm presentation is moved up to week 4, and the final presentation to week 9. The reports are actually collected about 1 week after the official end of the Summer registration period.

Week	Activity	Description
1 (1 - Summer)	Orientation	Students identify groups or indicate that they need one. All groups should be established before the next meeting.
2 (1 - Summer)	Project Identification	Groups identify the projects they are considering. Ideas are discussed and assignments to supervisors are made.
3 (2 - Summer)	Proposal Submission	Each group submits its proposal to its supervisor.
5 (4 - Summer)	Midterm Presentations	Each group makes a presentation of what its project is about, what the objectives are, and what the current progress is.
12 (9 - Summer)	Final Presentation and Demonstration	Each group presents its project and gives a demonstration of its operation.
13 (10 - Summer)	Report Submission	All project reports are to be turned in one week after the final presentations have been made.

Table 1: Schedule of Meetings and Activities throughout the semester.

2 Assessment

These are the components to the raw project grade.

1. The midterm presentation is worth 10% of the total grade.
   It is further broken down as follows:
   5% for how clearly the project is defined
   5% for the progress towards a solution

2. The final presentation is worth 15% and is broken down as follows:
   10% for the quality of the presentation
   5% for the usability of the software developed

3. The demonstration is worth 15% and is broken down as follows:
   10% for the amount of the project that is functional
   5% for the usability of the software developed
   There is a subjective level of difficulty rating that each grader assigns to each project. This rating is on a scale of 1 to 3, 1 being the least difficult and 3 the most difficult. The rating is used to adjust the contribution of the quality of the presentation and the

functionality. A project with a difficulty rating of 1 will have 5% contribution from the quality of the presentation and 15% contribution from the functionality. A project with a difficulty rating of 3 will have it the other way around: 15% from the presentation and 5% from the functionality. The rationale behind this is that a more ambitious (therefore difficult) project will have a lower chance of fully working, and therefore would have been more heavily penalised if the functionality counted for 10%. On the other hand, any lack of functionality will have to be compensated for by a good presentation. (Suggestions for improvements on this modifier scheme are welcome).

4. The final report is worth 50% of the total grade and it is divided as follows:

   10% for the quality of language used: this is a technical document and the language used should be appropriate. Proper English is expected.

   10% for the problem definition and investigation: it should be clear what the project ought to do, why it is useful and what the context of its development is (i.e. what are the relevant technologies to the project, and to what extent were they used?).

   15% for the description of the solution or implementation: this includes the design decisions that were made, and any clever ideas that were brought to bear in the implementation
   of the project.

   15% for the functionality, results and analysis: the degree to which the project was a success, how thoroughly the original problem was assessed, and what improvements could have been made to the solution's implementation with the benefit of hindsight.

5 Finally, each group is asked to build a website to "market" their project to the layman. This is assessed, and is worth 10% of your overall grade. All other aspects of the project ask you to describe what you have accomplished in the language of the computing discipline, where the expected audience is comprised of persons with a computing background. The website, on the other hand, is intended to allow anyone to get an appreciation for what you have accomplished. We make the website for each group publicly accessible, and actually direct visitors who are interested in what our students are capable of to it. This is one of your best opportunities to shine, and we encourage you to put effort into making your website attractive and interesting.

2.1 Student Assessment

At the end of the project, each student assesses himself and his other group members in each of three categories: programming, write-up and general participation. For each category, each group member has 12 points that he/she must distribute among all group members, including himself/herself. Note that the total number of points given by each group member for each category is a constant 12. The total number of points given to each member (including those given by himself) are then totaled to produce a peer-assessed weight. In any sized group (2 to 4), if each member gives all members an equal distribution of his 12 points for each category, then the total given to a single member will total 36 points (12 for each category).

Each supervisor is asked to indicate the degree to which he agrees with the peer assessment. The supervisor will make this assessment based on his interaction with the group during their regular meetings throughout the semester. The degree of supervisor agreement is translated into a proportion of the group grade (up to 30%) that will be modified by the peer-assessed weight.

For an individual with peer-assessed weight, w, in a group with modifier fraction m, and a raw group total score of s, his final score is computed using the formula:

$$total = \frac{w}{36} ms + (1 - m)s \quad (1)$$

For example, suppose that the members of a group of 3 have allotted their points as follows:

Member	Wts Assigned By Member 1			Wts Assigned By Member 2			Wts Assigned By Member 3			Total Weight
	Prog	Writ.	Particip	Prog	Writ.	Particip	Prog	Writ.	Particip	
1	6	4	4	5	3	4	6	4	4	40
2	3	4	4	4	5	4	3	4	4	35
3	3	4	4	3	4	4	3	4	4	33

Notice that each column sums to 12, which represents the total number of points that each member had to give out. In this example, member 1 has clearly done most of the programming, though the members disagree slightly on the relative excess that member 1 has contributed to the programming. Note also that in this example, all three members agree that all three of them participated equally well overall (group meetings, problem analysis, design specification, brainstorming, etc). Both members 1 and 3 thought that all members contributed equally to the writeup, but member 2 thought that he contributed a tad more than his group members to the write up. The total weights are shown to be 40, 35 and 33, respectively, for members 1, 2 and 3. Say that the raw project grade is 72%, and the modifier fraction (as determined by the supervisor) is 0.25. Then member 1 will actually get 40/36 * .25 * 72 + .75 * 72 = 20 + 54 = 74% and member 3 will get 33/36 * .25 * 72 + .75 * 72 = 16.5 + 54 = 70.5%. If you follow this calculation, you should have no problems determining that member 2 would have gotten 71.5%.

The course coordinator, who is responsible for final grades, reserves the right to modify a student's assessment or a supervisor's modifier if they are deemed to be unreasonably harsh towards another student (e.g. causing one student to marginally fail, when the group's grade was well above the pass mark). This right will be exercised only in extreme circumstances though, so all students should treat the peer assessment exercise with the care and integrity that it deserves.

3 The Presentations

The midterm presentation is only about 15 minutes long. In that time, you should focus on making a clear statement of what the project is about. In particular, it is important for you to be able to speak to the scope of the problem you are working on (what will you definitely not be doing, what will you be definitely doing, and what might you be doing). You should try to minimize the uncertainty in what you plan to do.

The second aspect of your midterm presentation should address the challenges that you expect to face. This is where you should bring out why the project is non-trivial. Sometimes it is useful to ask yourself "What about this project would prevent a group of average high-school

Computer Science students from being able to do it?" If you cannot identify something, then your project is probably aiming too low. At the end of the day, we want to see that you have brought knowledge from the courses in the curriculum to bear on the problem that you are addressing. **Most of all, though, the project should be something that you can be proud of when you have accomplished what you set out to do.**

Another aspect of the midterm presentation is your progress to date. We do not expect that you would have had much to show for your time up to that point, so we are not looking for examples of running code. However, we do want to know that you have given the implementation enough thought to have been able to identify the areas that might pose a technical challenge. You should note here that your having to learn a programming language is not quite the kind of technical challenge that we are looking for. The aim of asking you to consider the challenges is to get you to engage in problem solving activities where you might exercise knowledge gained from the rest of the curriculum (or elsewhere); the aim is not to get you to produce a list of your personal limitations.

3.1 The Final Presentation

The final presentation is expected to be polished. It should take about 15 minutes. During your presentation, you may show screenshots of your project to illustrate certain aspects of it, but do not interrupt your presentation to run your demonstration, unless it can be done quickly and smoothly (e.g. as a movie clip). You will have an opportunity to show how your project works and how it is used in the demonstration session(s). After your presentation, there will be a question and answer session, lasting no longer than 10 minutes.

Here are some tips for giving your final presentation:

- In giving your presentation, you should be very careful to say what was accomplished within the first 3 slides.

- Sometimes it helps to produce a screenshot of what was accomplished to illustrate what was intended.

- Rehearse your presentation, try to fit it within the given time limits.

- Do not be embarrassed about self-imposed limitations on the scope of the project. It is better to have a project that works that could have done more still, than to have a project that tried to do everything, but failed to do even one.

- Be excited about your work, and present it in as positive a light as possible. Even if your project does not fully work, emphasize the things that do work, and celebrate them. You should also deal with the parts that do not work, but do not let them dominate your presentation. This also implies that when you are still developing your solution, and you realize that you may not complete everything, you should focus on getting something working that is easy to show.

3.2 The Demonstration

The demonstration will be assigned to a day in the lab; this will be either the same day as the presentations or the following day. Groups are expected to setup their presentations on computers in the lab (or on their own computers that they will setup in the lab) for a specified period throughout the day. Visitors may be invited to view the demonstrations and interact with the group members. Assessors will visit each group at some time during the lab demonstrators to make their assessments.

Here are a few tips for presenting your demonstrations:

- Plan your demonstration. Do not simply do a breadth-first traversal of every feature that you can remember is implemented, or that is visible on the user interface.

- Think about the critical functionalities of your project, and try to show those first. Be prepared to show minor bits of functionality in case you are asked, but do not lead your demonstration with those.

- Do not demonstrate anything that you have not verified before (unless you cannot help it, e.g. when data is supplied by someone in the audience). You might be surprised to learn how software can fail in ways that you never anticipated, especially when the need for it to work is high.

- Make sure that you have all the necessary equipment in place in order to demonstrate your project. For example, if you have implemented a project that requires special hardware, or specially configured computers, do not come expecting to find them in the lab. We might be able to accommodate some requests for infrastructural support, but you should not count on it. If your project has special needs that make the lab an unsuitable place to demonstrate then try to make an arrangement with the assessors (through the course coordinator) in advance of the demonstration date.

4 The Final Report

The final report is supposed to be a technical document that presents the work done by a group. It should be thorough, but not long-winded. I expect about 20 pages with a reasonable font size (11 or 12) and reasonable spacing (single spacing). It should not exceed 35 pages without good reason. Nonetheless, the number of pages in the document is not an important consideration in the assessment of the document.

The final report should be broken down in approximately five sections:

Introduction: Present the problem being solved, why it is useful or important, and an overview of what you managed to accomplish for your project.

Background: Describe the existing relevant technologies that impacted (or ought to have impacted) your implementation. Also include mention of any products or tools that are similar to what your project aimed to accomplish.

Method: Present your design (and architecture), highlighting its positive features; discuss the tradeoffs that were made and the reasoning behind some of the decisions that were not completely constrained; describe the implementation of the design, including any technical challenges that arose, and how they were surmounted. Describe how testing was done, and if any automated evaluation was done, how it was carried out. Describe your design, and what tradeoffs had to be made when creating it. How would your design scale to larger problems if your project were expanded beyond its original scope?

Results: Present data that measures various aspects of your implementation. Use the objectives that you used in the definition of the project as your benchmark for assessing the degree of success of your project. This is where you would present screenshots of the user interface at significant points in the program. If performance is an issue or if the quality of your implementation can be measured quantitatively, you would also present data that reports on that in this section.

Conclusions: Give an assessment of the success of your implementation; to what degree was the original problem described solved? Given all that you have found in the results section, and those decisions that were made in the Method section, are there things that would have been done differently? how so? What lessons can be taken away from the implementation of the project (e.g. don't try to use a certain technology because it is too hard to debug, or use a particular programming technique because it is highly effective)? What extensions could be made to the solution or what modifications could have been made to the original problem definition to produce a better degree of success?

Along with your reports, you should also submit softcopies of the software you have developed (preferably on CD-ROM). Since there are no other means by which this course is assessed, you will not be given back your projects. You should take the opportunity to examine your report when it is graded, and see the comments written by your supervisor. One positive consequence of this is that you have available for your perusal, the project reports that were submitted in the past – complete with grades and comments. You are encouraged to examine these before your write your own reports so that you have a good idea of what is expected of you.

5 Closing Remarks

It is worthwhile mentioning that the project course should really represent for you, the student, an opportunity to express yourselves in a creative way with computation. Although we do hold you to aiming to produce at or above a minimum threshold of complexity, we do want you to think broadly and to pursue projects that you can become passionate about. We especially would like to see students taking ownership for their projects and presenting them with pride. We want to be proud of what our students are capable of producing, and this is more likely to happen if

you are passionate about your work. We hope that you will find COMP3901 to be one of your most enjoyable courses by the time you are finished with it.